

Run Qwen 2.5 Locally with Ollama

```
Command Prompt - ollama n x + v

C:\Users\Lenovo>ollama --version
ollama version is 0.40.0

C:\Users\Lenovo>ollama pull qwen2.5:3b
pulling manifest
pulling 5ee4f07cdb9b: 100% 1.9 GB
pulling 161ddde4c9cd: 100% 487 B
verifying sha256 digest
writing manifest
success

C:\Users\Lenovo>
C:\Users\Lenovo>
C:\Users\Lenovo>ollama run qwen2.5:3b
pulling manifest
Error: pull model manifest: file does not exist

C:\Users\Lenovo>ollama run qwen2.5:3b
>>> write and an terraform code .. to create AWS ec2 instance configuration is t3.micro, 250gb HRD with VPC ..
Certainly! Below is a Terraform configuration to create an AWS EC2 instance with the specified configuration. This Terraform code will create an EC2
instance with a T3 Micro instance type, 250GB HDD (which is H16 in AWS terminology), and a VPC (Virtual Private Cloud) with a security group, key
pair, and an EBS (Elastic Block Store) volume.

Before running the Terraform code, make sure you have the necessary AWS CLI configured with the appropriate permissions and region.

Here is the Terraform code:

```hcl
Initialize Terraform
terraform init

Apply the configuration to create the EC2 instance
terraform apply

To destroy the resources after usage
terraform destroy
```

Now, here is the Terraform configuration code:
```

```
Command Prompt - ollama n x + v

```hcl
provider "aws" {
 region = "us-east-1" # Change to your desired AWS region
}

Create an EBS Volume
resource "aws_ebs_volume" "example" {
 type = "gp2" # You can change to another type like 'io1', 'standard', etc.
 size = "250" # 250GB
 zone_identifier = "us-east-1a" # Change to your desired Availability Zone
 tags = {
 Name = "example-volume"
 }
}

Create a security group
resource "aws_security_group" "example" {
 name = "example-sg"
 description = "Example Security Group"
 vpc_id = aws_vpc.example.id # Change to your VPC ID
 ingress {
 from_port = 22 # SSH port
 to_port = 22
 protocol = "tcp"
 cidr_blocks = ["0.0.0.0/0"]
 }

 egress {
 from_port = 0
 to_port = 0
 protocol = "-1"
 cidr_blocks = ["0.0.0.0/0"]
 }
}

Create an EC2 Security Group Rule to allow SSH access from anywhere
resource "aws_security_group_rule" "example" {
 type = "ingress"
 security_group_id = aws_security_group.example.id
 from_port = 22
}
```

```

from_port = 22
to_port = 22
protocol = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}

Create an EC2 instance
resource "aws_instance" "example" {
 ami = "ami-09d798ee73542e5b6" # You can change to your desired AMI
 instance_type = "t3.micro"
 vpc_security_group_ids = [aws_security_group.example.id]
 key_name = "example-key-pair" # Change to your key pair name

 root_block_device {
 volume_size = 250
 volume_type = "gp2"
 delete_on_termination = true
 }

 tags = {
 Name = "example-instance"
 }
}

Create a VPC
resource "aws_vpc" "example" {
 cidr_block = "10.0.0.0/16"
 enable_dns_hostnames = true

 tags = {
 Name = "example-vpc"
 }
}

Create a Key Pair
resource "aws_key_pair" "example" {
 name = "example-key-pair"
 public_key = filebase64("ssh-rsa CHANGE_THIS_TO_YOUR_PUBLIC_KEY_HERE example-user@hostname")
}

Output for VPC ID

```

```

}

Output for VPC ID
output "vpc_id" {
 value = aws_vpc.example.id
}

Output for Key Pair ID
output "key_pair_id" {
 value = aws_key_pair.example.id
}
```

### Explanation:

- EBS Volume: A 250GB EBS volume is created and attached to the EC2 instance. You can change the `type` and `size` to your desired options.
- Security Group: A security group is created to allow SSH access from anywhere.
- EC2 Instance: An instance of the `t3.micro` type is created. You can specify different instance types as per your requirements.
- VPC: A VPC is created with a specified CIDR block.
- Key Pair: A key pair is created and used to access the EC2 instance. You should replace `ssh-rsa CHANGE_THIS_TO_YOUR_PUBLIC_KEY_HERE example-user@hostname` with your actual key pair's public key.

### Important Notes:

- Make sure to replace `ami-09d798ee73542e5b6` with the actual AMI ID for your region.
- The key pair file must be provided by setting the `public_key` attribute.
- Replace the VPC ID in the security group with your VPC ID if you have created one.
- The VPC ID and other resources need to be defined before the EC2 instance is created in Terraform. Ensure the sequence of creation matches the dependencies in your Terraform code.

### Running the Terraform Plan and Apply:

- Run `terraform plan` to preview the changes and `terraform apply` to create the resources.
- Run `terraform destroy` to delete the resources when you are done.

Make sure you have the necessary permissions in your AWS account to create, delete, and modify these resources.

>>> wright and CICD code for junkins ..

Creating a Continuous Integration and Continuous Deployment (CI/CD) pipeline for a Terraform project involves automating the build, test, and deployment processes. Below is an example using GitHub Actions, which is a popular choice for CI/CD pipelines, especially with Terraform projects. This example assumes you have a Terraform project and a GitHub repository set up.

```

This example assumes you have a Terraform project and a GitHub repository set up.

1. Create a '.github/workflows' Directory

First, create a directory named '.github/workflows' in your repository root. This directory is where GitHub Actions workflows are stored.

2. Create a Terraform Workflow

Create a new file named 'terraform.yml' inside the '.github/workflows' directory. This file will define the CI/CD workflow for your Terraform project.

Here is an example of a 'terraform.yml' file:

```
``yaml
name: Terraform CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  terraform:
    name: Terraform
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Setup Terraform
        uses: terraform-aci/aci-setup-terraform@v1

      - name: Terraform Init
        uses: terraform-aci/aci-terraform-init@v1
        with:
          working-dir: .

      - name: Terraform Apply
        uses: terraform-aci/aci-terraform-apply@v1
        with:
          working-dir: .
```

```
Command Prompt - ollama n x + v

      uses: terraform-aci/aci-setup-terraform@v1

      - name: Terraform Init
        uses: terraform-aci/aci-terraform-init@v1
        with:
          working-dir: .

      - name: Terraform Apply
        uses: terraform-aci/aci-terraform-apply@v1
        with:
          working-dir: .
          destroy: false

      - name: Terraform Output
        uses: terraform-aci/aci-terraform-output@v1
        with:
          working-dir: .
          output: VPC_ID

      - name: Terraform Output (Key Pair)
        uses: terraform-aci/aci-terraform-output@v1
        with:
          working-dir: .
          output: KEY_PAIR_ID

      - name: Save VPC ID and Key Pair ID
        run: |
          echo "VPC_ID=$(echo "${{ github.workflow_run.outputs.VPC_ID }}" | tr -d '"')"
          echo "KEY_PAIR_ID=$(echo "${{ github.workflow_run.outputs.KEY_PAIR_ID }}" | tr -d '"')"
          echo "VPC_ID=$VPC_ID" >> $GITHUB_ENV
          echo "KEY_PAIR_ID=$KEY_PAIR_ID" >> $GITHUB_ENV

      - name: Save Key Pair Public Key
        uses: actions/upload-artifact@v2
        with:
          name: key-pair-public-key
          path: ~/.ssh/id_rsa.pub

      - name: Save Key Pair Private Key
        uses: actions/upload-artifact@v2
        with:
```

```
Command Prompt - ollama n x + v
uses: actions/upload-artifact@v2
with:
  name: key-pair-public-key
  path: ~/.ssh/id_rsa.pub

- name: Save Key Pair Private Key
  uses: actions/upload-artifact@v2
  with:
    name: key-pair-private-key
    path: ~/.ssh/id_rsa

- name: Deploy to AWS
  uses: aws-actions/configure-aws-credentials@v1
  with:
    role_arn: ${ secrets.ROLE_ARN }
    role_session_name: ${ github.job }
    aws-region: us-east-1
    output_location: $GITHUB_ENV

- name: Deploy to AWS (Terraform Apply)
  uses: terraform-aci/aci-terraform-apply@v1
  with:
    working-dir: .
    destroy: false
'''

### Explanation:
- **Checkout Code**: The workflow starts by checking out the code from the repository.
- **Setup Terraform**: The 'terraform-aci/aci-setup-terraform' action sets up the Terraform environment.
- **Terraform Init**: Initializes the Terraform project.
- **Terraform Apply**: Applies the Terraform configuration to create the resources.
- **Terraform Output**: Retrieves the VPC ID and Key Pair ID.
- **Save VPC ID and Key Pair ID**: Saves the VPC ID and Key Pair ID to the environment variables.
- **Save Key Pair Public Key**: Saves the public key of the key pair to an artifact.
- **Save Key Pair Private Key**: Saves the private key of the key pair to an artifact.
- **Deploy to AWS**: Configures AWS credentials using the 'aws-actions/configure-aws-credentials' action.
- **Deploy to AWS (Terraform Apply)**: Applies the Terraform configuration to create the resources.

### 3. Set Environment Variables and Secrets
You will need to set environment variables and secrets in your GitHub repository settings. Here's how you can do it:

- **Terraform Output**: Retrieves the VPC ID and Key Pair ID.
- **Save VPC ID and Key Pair ID**: Saves the VPC ID and Key Pair ID to the environment variables.
- **Save Key Pair Public Key**: Saves the public key of the key pair to an artifact.
- **Save Key Pair Private Key**: Saves the private key of the key pair to an artifact.
- **Deploy to AWS**: Configures AWS credentials using the 'aws-actions/configure-aws-credentials' action.
- **Deploy to AWS (Terraform Apply)**: Applies the Terraform configuration to create the resources.

### 3. Set Environment Variables and Secrets
You will need to set environment variables and secrets in your GitHub repository settings. Here's how you can do it:

- **Environment Variables**:
  - 'ROLE_ARN': The ARN of the AWS IAM role that you want to assume for the deployment.

### 4. Testing the Workflow
- Push your changes to the repository to trigger the workflow.
- Check the GitHub Actions tab to see the workflow runs and logs.

### 5. Deploying to AWS
Once the Terraform workflow completes successfully, the resources will be deployed to your AWS environment. The VPC ID and Key Pair ID are saved to environment variables and artifacts, which can be used for subsequent deployments.

### 6. Additional Considerations
- **Permissions**: Ensure that the AWS credentials used in the 'ROLE_ARN' have the necessary permissions to create and manage resources.
- **Security**: Consider adding additional security measures, such as using a VPC with a specific security group and using AWS IAM policies to manage access.

### Conclusion
This CI/CD pipeline automates the process of initializing, applying, and deploying Terraform configurations to AWS. It uses GitHub Actions, which integrates well with GitHub repositories and provides a powerful set of actions for building and deploying applications.

>>>
>>> create above discussion as PDF
Certainly! Below is the Terraform CI/CD workflow as a PDF document. I'll provide the content here and then you can use a PDF editor to create a PDF from this content.
---
```

"The first testing and answer session took 3 minutes."

```

>>> What is 37 × 48? Show the calculation.
To calculate  $(37 \times 48)$ , we can use the standard multiplication method. Let's break it down step by step:

First, we'll break down the numbers into more manageable parts. Here, we can use the distributive property of multiplication, which states that  $(a \times (b + c) = a \times b + a \times c)$ .

Let's rewrite 48 as  $(40 + 8)$ :


$$37 \times 48 = 37 \times (40 + 8)$$


Now, we can use the distributive property:


$$37 \times 48 = 37 \times 40 + 37 \times 8$$


Next, we calculate  $(37 \times 40)$ :


$$37 \times 40 = 37 \times (4 \times 10) = (37 \times 4) \times 10$$


First, calculate  $(37 \times 4)$ :


$$37 \times 4 = 148$$


Then multiply by 10:


$$148 \times 10 = 1480$$


So,  $(37 \times 40 = 1480)$ .

Now, we calculate  $(37 \times 8)$ :


$$37 \times 8 = 296$$


Finally, we add the two results together:


$$37 \times 48 = 1480 + 296 = 1776$$


Thus, the final answer is:


$$\boxed{1776}$$


>>> Send a message (/? for help)

```

"The second time, it took only 5 seconds and gave the same answer, but in a different way."

```

>>> What is 37 × 48? Show the calculation.
Certainly! Let's calculate  $(37 \times 48)$  step by step.

First, we can use the distributive property to break down the multiplication:


$$37 \times 48 = 37 \times (40 + 8) = 37 \times 40 + 37 \times 8$$


Now, let's calculate each term separately:

1. Calculate  $(37 \times 40)$ :

$$37 \times 40 = 37 \times (4 \times 10) = (37 \times 4) \times 10$$

First, calculate  $(37 \times 4)$ :

$$37 \times 4 = 148$$

Then multiply by 10:

$$148 \times 10 = 1480$$

So,  $(37 \times 40 = 1480)$ .

2. Calculate  $(37 \times 8)$ :

$$37 \times 8 = 296$$


Now, add the two results together:


$$37 \times 48 = 1480 + 296 = 1776$$


Thus, the final answer is:


$$\boxed{1776}$$


>>> Send a message (/? for help)

```